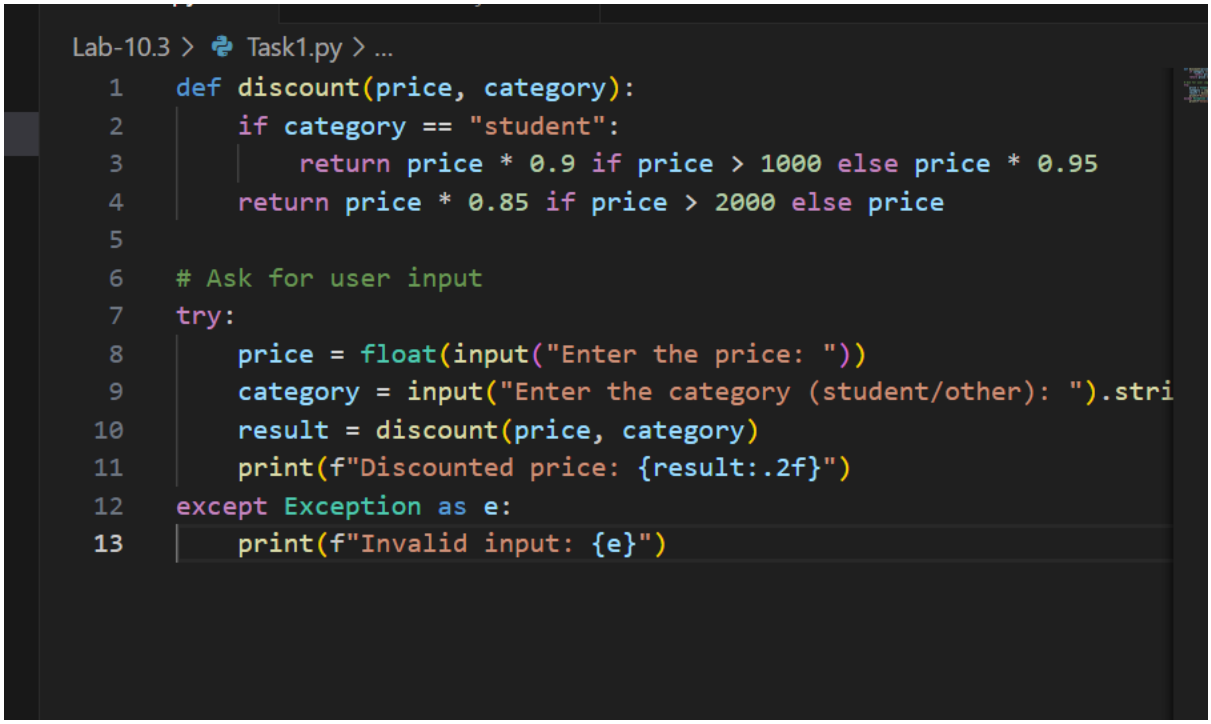
Lab-10.3

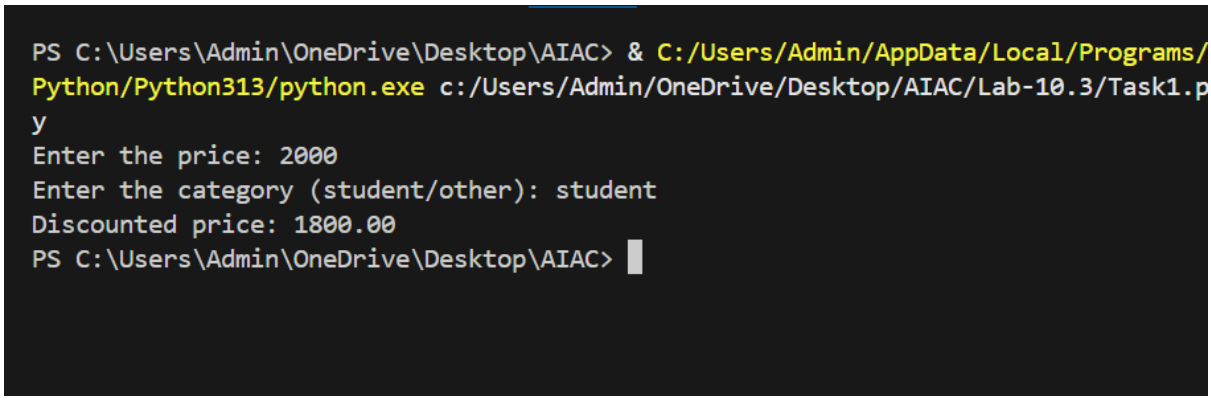
Task Description #1 – Refactor Nested Conditionals

Task: Provide AI with the following nested conditional code and ask it to simplify and refactor for readability.



```
Lab-10.3 > Task1.py > ...
1   def discount(price, category):
2       if category == "student":
3           return price * 0.9 if price > 1000 else price * 0.95
4       return price * 0.85 if price > 2000 else price
5
6   # Ask for user input
7   try:
8       price = float(input("Enter the price: "))
9       category = input("Enter the category (student/other): ").stri
10      result = discount(price, category)
11      print(f"Discounted price: {result:.2f}")
12  except Exception as e:
13      print(f"Invalid input: {e}")
```

OUTPUT:



```
PS C:\Users\Admin\OneDrive\Desktop\AIAC> & C:/Users/Admin/AppData/Local/Programs/
Python/Python313/python.exe c:/Users/Admin/OneDrive/Desktop/AIAC/Lab-10.3/Task1.p
y
Enter the price: 2000
Enter the category (student/other): student
Discounted price: 1800.00
PS C:\Users\Admin\OneDrive\Desktop\AIAC>
```

Task Description #2 – Optimize Redundant Loops

Task: Give AI this messy loop and ask it to refactor and optimize.

```python
def find_common(a, b):
    return list(set(a) & set(b))

# Ask for user input
try:
    a = input("Enter the first list of items (comma separated): "
    b = input("Enter the second list of items (comma separated):
    # Remove extra spaces
    a = [item.strip() for item in a]
    b = [item.strip() for item in b]
    result = find_common(a, b)
    print(f"Common elements: {result}")
except Exception as e:
    print(f"Invalid input: {e}")
```

OUTPUT:

```
y
Enter the first list of items (comma separated): varshini,rishitha
Enter the second list of items (comma separated): deekshitha,shreya
Common elements: []
PS C:\Users\Admin\OneDrive\Desktop\AIAC>
```

Task Description #3 – Improve Class Design

Task: Provide this class with poor readability and ask AI to improve:

- Naming conventions
- Encapsulation
- Readability & maintainability

-

```
-10.3 > Task3.py > ...
lass Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def increment(self, percent):
        self.salary += self.salary * percent / 100

    def display(self):
        print(f"Employee: {self.name}, Salary: {self.salary:.2f}")

ry:
    name = input("Enter employee name: ")
    salary = float(input("Enter current salary: "))
    percent = float(input("Enter increment percentage: "))
    emp = Employee(name, salary)
    emp.increment(percent)
    emp.display()
xcept Exception as e:
    print(f"Invalid input: {e}")
```

OUTPUT:

```
y
Enter employee name: varsh
Enter current salary: 5000
Enter increment percentage: 5
Employee: varsh, Salary: 5250.00
PS C:\Users\Admin\OneDrive\Desktop\AIAC>
```

Task Description #4 – Modularize Long Function

Task: Give AI this long unstructured function and let it modularize into smaller helper functions.

```
-10.3 > task4.py > ...
def process_scores(scores):
    avg = sum(scores) / len(scores)
    highest = max(scores)
    lowest = min(scores)
    print("Average:", avg)
    print("Highest:", highest)
    print("Lowest:", lowest)

try:
    scores = list(map(float, input("Enter scores separated by com
    process_scores(scores)
except Exception as e:
    print(f"Invalid input: {e}")
```

OUTPUT:

```
y
Enter scores separated by commas: 23,12,5,8
Average: 12.0
Highest: 23.0
Lowest: 5.0
PS C:\Users\Admin\OneDrive\Desktop\AIAC>
```

Task Description #5 – Code Review on Error Handling

Task: Provide AI with this faulty code and ask it to improve error handling, naming, and readability.

```python
def div(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Error: Division by zero"

try:
    a = float(input("Enter the numerator: "))
    b = float(input("Enter the denominator: "))
    print(div(a, b))
except Exception as e:
    print(f"Invalid input: {e}")
```

OUTPUT:

```
Enter the numerator: 225
Enter the denominator: 5
45.0
PS C:\Users\Admin\OneDrive\Desktop\AIAC>
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```python
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

try:
    score = float(input("Enter the score: "))
    print(f"Grade: {grade(score)}")
except Exception as e:
    print(f"Invalid input: {e}")
```

OUTPUT:

```
Enter the score: 90
Grade: A
PS C:\Users\Admin\OneDrive\Desktop\AIAC>
```